ClaimCheck: Product Spec Pack

PRD · TRD · App flow · UI/UX brief · Backend schema · Implementation plan

Status: Draft v1 · 21 Oct 2026 · Companion to "ClaimCheck: 1-Day Build Guide" (holds the code).

Preliminary decision support only. Not a binding insurer decision.

Security baseline: the 18 checks (exposed DB credentials … NoSQL injection) are built in as numbered requirements SEC-1 to SEC-18 (TRD section 12) and gate every phase of the plan.

Assumptions (change them and the docs change):

- Market: India. Currency INR. English documents only.
- Stack fixed by prior decision: Supabase, FastAPI, NVIDIA Nemotron, React + Vite.
- Team: 1 builder, 1 day for MVP, then hardening.
- No insurer integration exists.

Part A: PRD

1. Summary

User uploads a health-insurance policy PDF and a hospital bill, asks a question. ClaimCheck returns one of four verdicts (Covered, Partially covered, Not covered, Documents missing), each backed by a quoted policy clause and the bill lines it used. If evidence is weak, it retries retrieval a bounded number of times, then hands off to a human. It never invents clauses, lines, or amounts.

2. Problem

- Policy wordings are long. Clauses that matter (room-rent limits, co-pay, exclusions, waiting periods) are easy to miss.
- Bills have many lines. Matching each line to a clause by hand is slow and error-prone.
- People find out what is not covered only after a claim is filed.

3. Users

Persona	Need	When
Policyholder / family member	"Will this bill be paid? What will I owe?"	around hospital discharge
Hospital desk / claims helper	quick pre-check before submitting	per claim
Human reviewer (day 2)	handle flagged cases with evidence in front of them	daily queue

4. Goals and non-goals

Goals

- G1: Verdict with verifiable evidence (policy quote + bill lines).
- G2: Zero invented citations or amounts. Abstain instead.
- G3: Treat health documents as sensitive: private by default, isolated per user.
- G4: Working MVP in one day.

Non-goals (MVP)

- Binding claim decisions, filing claims, live claim-status checks (needs an authorised insurer integration)
- OPD, dental, maternity-specific logic, waiting-period computation
- Non-English or handwritten documents
- Reconciling several policies or several bills in one claim
- Native mobile apps, payments, admin console

5. User stories and acceptance criteria

ID	Story	Acceptance
US-1	As a user I sign up and sign in.	Email confirmed before first use. Wrong credentials give one generic message.
US-2	I upload a policy PDF and a bill (PDF or image).	Wrong type or oversize file rejected with a clear message. Progress shown.

ID	Story	Acceptance
US-3	I ask a question about the claim.	Question 1–500 chars. Empty or longer is rejected.
US-4	I get a verdict.	Exactly one of 4 verdicts, or "needs human review". Preliminary-assessment banner always visible.
US-5	I see the evidence.	Each policy citation shows clause ID, page, exact quote. Each bill line shows item, amount, page, source text.
US-6	I see amounts only when supported.	Eligible and non-eligible amounts appear only if computed from verified rules. Otherwise "Not calculable from supplied evidence".
US-7	I see what is missing.	`documents_missing` lists the documents needed.
US-8	I see my past claims, and only mine.	List shows own claims. Other users' ids return nothing.
US-9 (P1)	As a reviewer I open flagged claims and record an outcome.	Reviewer role checked server-side. Every view and decision is audit-logged.
US-10	I delete my documents.	Documents, chunks, items removed. Claims keep no document text.

6. Functional requirements

ID	Requirement	Pri
FR-1	Email + password auth via Supabase Auth	P0
FR-2	Upload policy and bill to a private bucket under the user's folder	P0
FR-3	Extract text with page numbers. OCR when no text layer	P0
FR-4	Split policy into clause chunks with page + clause ID	P0
FR-5	Extract bill line items. Each line must be found verbatim in the source text	P0

ID	Requirement	Pri
FR-6	Flag bill total mismatch and low OCR confidence. Never auto-correct amounts	P0
FR-7	Hybrid retrieval: keyword + vector, merged	P0
FR-8	CRAG loop, max 2 retries, then abstain	P0
FR-9	Extract rules (exclusion, co-pay %, room-rent cap, proportionate deduction) with verbatim quotes	P0
FR-10	Reject any rule whose quote is not in a retrieved clause on the cited page	P0
FR-11	Compute amounts in code with `Decimal`. LLM never calculates	P0
FR-12	Map internal status to one of 4 verdicts or human review	P0
FR-13	Result page with evidence, missing documents, limitations	P0
FR-14	Claim history (own claims only)	P0
FR-15	Per-user rate limit on analyze	P0
FR-16	Delete own documents	P1
FR-17	Reviewer queue, notes, outcome	P1
FR-18	Retention job	P1
FR-19	Reviewer MFA	P1
FR-20	`check_claim_status`	P2: only with authorised insurer integration. Until then the product says "cannot check status". Never simulate.

7. Non-functional requirements

ID	Requirement
NFR-1	Analyze target under 30 s for a typical case (proposal. Measure, then set the real target).
NFR-2	Limits (proposal): 10 MB per file, 30 pages per document, 500-char question, 10 analyses per user per hour.
NFR-3	No document text in logs, error bodies, or traces.
NFR-4	Accessible: WCAG 2.1 AA target. Never colour-only status.
NFR-5	Works on a 375 px wide phone.
NFR-6	All 18 security checks pass (TRD section 12).

8. Success measures

Measure	Gate / target
Calculation accuracy on unit tests	100%
Fake-citation rejection	100%
Prompt-injection fixture: verdict unchanged	pass
Cross-user access tests	all blocked
Verdict accuracy, OCR amount match, Recall@5, abstention quality, human-review rate	measure baseline on fixtures, then set targets. No invented numbers.

9. Scope by release

Release	Contents
MVP (day 1)	FR P0
Hardening (days 2–5)	FR P1, rate-limit tuning, table-aware parsing, more rule types, eval set to 30+ cases

Release	Contents
Later	P2, more languages, OCR upgrade, queue worker

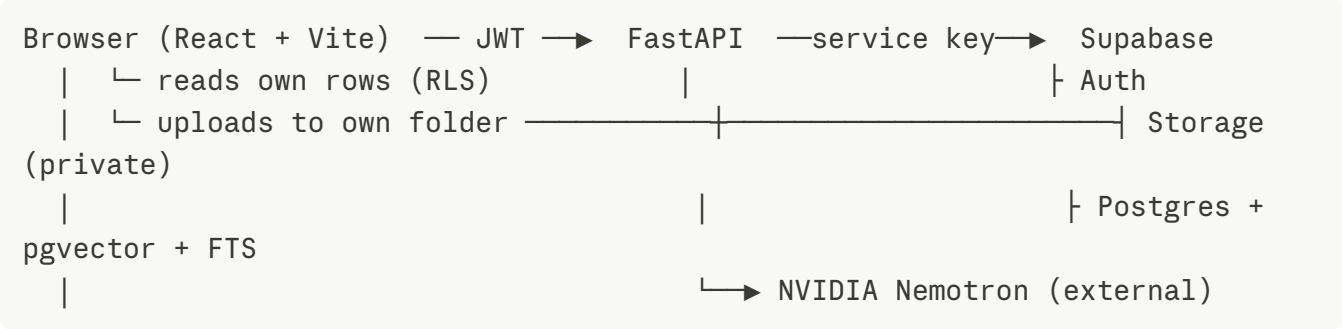
10. Risks and open questions

Risk	Mitigation
Medical text sent to third-party LLM	Read NVIDIA data terms. Mask identifiers. Consent notice. Consider self-hosting.
Grader says "sufficient" wrongly	No verified rule → forced human review. Spot-check every release.
Policy wording varies	Rules come from the clause text. Unclear → human review.
Users treat output as final	Banner + wording on every screen.
Data-protection law applies to health data	Check DPDP Act obligations with counsel before real users. Not legal advice.

Open questions: retention period? who are the human reviewers? is self-hosted LLM required? which insurers' wordings to support first?

Part B: TRD

1. Architecture



Trust zones: browser untrusted · FastAPI trusted (holds service key, bypasses RLS) · Supabase enforced by RLS · NVIDIA third party.

2. Stack

Layer	Choice	Note
Frontend	React + Vite + TypeScript + Tailwind	static build on Vercel/Netlify
Auth	Supabase Auth	JWT verified by backend via Supabase
API	FastAPI (Python 3.12)	Pydantic validation
DB	Supabase Postgres, pgvector (384-d), FTS	one DB, no second vector store
Files	Supabase Storage, private bucket	
OCR	PyMuPDF + Tesseract	
Embeddings	bge-small-en-v1.5 via fastembed	CPU
LLM	NVIDIA Nemotron, OpenAI-compatible API	confirm model ID
Host (API)	Docker on Render/Railway/Fly	confirm current plan limits

Pin every version after the first working install. Commit lockfiles.

3. Modules

Module	Job
`main.py`	routes, auth dependency, CORS, error handler
`parse.py`	text/OCR, clause split, bill structuring, file-type check
`retrieve.py`	embed, hybrid search RPC
`agent.py`	CRAG loop, grading, rule extraction, citation check, verdict
`rules.py`	money math (`Decimal`)
`llm.py`	JSON-validated model calls
`config.py` , `db.py`	settings, Supabase client

Code: see the Build Guide, sections 4 and 14 (14 overrides 4.4–4.7).

4. API

All routes except `/health` need `Authorization: Bearer <JWT>`. IDs are UUIDs. Errors use one shape: `{"error":{"code":"...","message":"...","request_id":"..."}}`. No stack traces, ever.

Route	Request	Success	Errors
`GET /health`	none	`{ok:true}`	
`POST /documents/{id}/process`	path id	202 `{status:"processing"}`	401, 404 (not yours or not found), 409 (already processed), 422
`POST /claims/analyze`	`{policy_id, bill_id, question}`	200 claim row with `result`	401, 404, 409 (doc not ready), 422, 429 (rate limit), 502 (LLM down)
`DELETE /documents/{id}` (P1)	path id	204	401, 404
`GET /reviews/queue` (P1, reviewer)	none	flagged claims	401, 403
`POST /reviews/{claim_id}` (P1, reviewer)	`{outcome, notes}`	200	401, 403, 404, 422

Reads of `documents` and `claims` go straight to Supabase from the browser under RLS. No `GET /claims` route.

Polling: browser polls `documents.status` every 2 s, gives up after 3 min.

5. Processing pipeline

1. **Upload**: browser → Storage at `<uid>/<uuid>.<ext>`, inserts `documents` row (`status=uploaded`).
2. **Process**: backend downloads the object, checks magic bytes (`%PDF`, PNG, JPEG), enforces page cap, extracts text per page (OCR when empty), then:

- policy: split into clause chunks, embed, insert.
- bill: redact identifiers, LLM extracts lines, code drops any line not found verbatim, checks sum vs printed total, flags low OCR confidence.

3. **Analyze**: retrieve → grade → rewrite/retrieve (max 2 retries) → rule extraction → citation validation → `rules.calc` → verdict mapping → store claim with trace.
4. **Show**: browser reads the claim row.

Verdict mapping

Internal state	Output
Evidence sufficient, rules verified, computed	`covered` if eligible = claimed; `not_covered` if eligible = 0; else `partially_covered`
Evidence sufficient but no verified rule	amounts unadjusted, `human_review_required = true`
Insufficient and grader lists missing documents	`documents_missing`
Insufficient, no document named, retries exhausted	no verdict, abstain, human review
No bill lines extracted	no verdict, human review
Low OCR confidence or total mismatch	verdict allowed, `human_review_required = true`, limitation shown

6. LLM contract

Task	Input	Output (Pydantic)
Bill structuring	one page of OCR text	`{items: [{item,quantity,unit_price,amount,source_text}], total}`
Grade evidence	question, bill item names, clauses	`{relevant, sufficient, missing, missing_docs[]}`
Rewrite query	question, missing concept	`{query}`

Task	Input	Output (Pydantic)
Extract rules	bill item names, clauses	`{rules: [{kind,value,applies_to,clause_id,page,quote}]}`

Rules: temperature 0 · 60 s timeout · 3 attempts with backoff · reply must parse as JSON and validate or the call fails · document text is wrapped in tags and the system prompt says it is data, never instructions · the model has no tools and cannot call anything · the model never does arithmetic.

7. Citation and abstention rules

- A rule is kept only if its `quote` appears (whitespace-normalised) in a retrieved chunk on the same page.
- Bill lines are kept only if `source_text` appears in the page text sent to the model.
- Retry limit configurable, default 2. After that: abstain. No further calls.

8. Configuration and secrets

Name	Lives in	Public?
`SUPABASE_URL`	backend env, frontend env	yes
Publishable key	frontend env (`VITE_`)	yes (by design)
`SUPABASE_SERVICE_KEY`	backend host secret store only	NEVER
`NVIDIA_API_KEY` , `NVIDIA_MODEL`	backend host secret store only	NEVER
`FRONTEND_ORIGIN`	backend env	no

9. Observability

Structured logs with request ID, route, status, duration, user ID hash, counts (chunks, items, LLM calls, retries). Never document text, quotes, amounts, tokens, or keys. Alerts: error rate, LLM failures, 429 spikes, stuck `processing` documents.

10. Deployment and CI

- Environments: dev (local + dev Supabase project), prod (separate Supabase project and keys).
- CI steps: install from lockfiles → `gitleaks` → `pip-audit` / `npm audit` → tests → build.
- CI logs must not print env. No `printenv`, no `set -x`. Secrets only via the CI secret store.
- Docker image built from a minimal base. `.dockerignore` excludes `.env`, `.git`, `tests/external/`.
- HTTPS enforced by both hosts. Frontend host sends security headers (CSP, `X-Content-Type-Options`, `Referrer-Policy`, frame protection). Verify with a header scanner.
- Prod runs without `--reload`. API docs routes disabled.

11. Testing

Layer	What
Unit	`rules.calc`, `validate_citations`, clause split, PII redaction
Fixtures	6 generated cases with known answers (Build Guide section 17)
Security	cross-user tests, bypass tests, SEC-1..18 checks below
Retrieval	Recall@5 on labelled questions
Manual	blurry scan, long policy, odd bill layouts

12. Security requirements (the 18 checks)

ID	Check	Control in this system	Verify
SEC-1	Exposed DB credentials	No direct DB connection from app code. Backend talks to Supabase over HTTPS with the service key. DB password stays in the Supabase dashboard, rotated after setup.	Grep repo and host env for `postgres://` and the DB password.
SEC-2	Public `.env` files	`.env` in `.gitignore` and `.dockerignore`. Hosts use secret stores. Static host serves `dist/` only.	`curl /.env` on both domains → 404. List image files → no `.env`.
SEC-3	Hardcoded API keys	Keys only through `pydantic-settings` env. `gitleaks` in pre-commit and CI.	gitleaks clean. Grep for key prefixes.
SEC-4	Weak authentication	Supabase Auth. Email confirmation on. Minimum password length set. Leaked-password protection if the plan offers it (check). Built-in auth rate limits on. Reviewer MFA (FR-19).	Weak password rejected. Unconfirmed email blocked.
SEC-5	Missing authorization checks	Every route verifies the JWT, then checks ownership of every id. Reviewer routes check a `reviewers` table server-side.	Tests: other user's ids → 404. Normal user on reviewer route → 403.
SEC-6	Broken data isolation	RLS on all tables. Backend filters `user_id` on every query. Storage policy on folder = `auth.uid()`.	User B tests (Build Guide 6.3).

ID	Check	Control in this system	Verify
SEC-7	Open database permissions	`anon`: nothing. `authenticated`: select on own rows, insert on `documents` only. `hybrid_search` revoked from browser roles. No `SECURITY DEFINER` in public.	`supabase db advisors` clean. Anon key select → denied.
SEC-8	Misconfigured storage	Private bucket. File-size and mime limits on the bucket. Folder policy. No public URLs.	Anonymous GET of an object → denied. User B download → denied.
SEC-9	Unprotected admin routes	No admin UI in MVP. Reviewer routes (P1): role from `reviewers` table, never `user_metadata`. Each access audit-logged.	403 tests. Audit rows exist.
SEC-10	Exposed debug tools	FastAPI `docs_url`, `redoc_url`, `openapi_url` set to `None` in prod. No `--reload`. No debug flags.	`curl /docs`, `/openapi.json` → 404.
SEC-11	Leaking build logs	CI masks secrets. No env dumps. No secrets as Docker build args.	Read the last CI logs. Search for key prefixes.
SEC-12	Verbose error messages	Global exception handler returns generic message + request ID. Details logged by exception type only.	Force an error → no traceback or path in the response.
SEC-13	Secrets in Git history	`gitleaks detect` over full history before the repo is shared. Any hit → rotate the key. Deleting the commit is not enough.	gitleaks history scan clean.
SEC-14	Frontend secrets	Frontend env holds only Supabase URL, publishable key, API URL.	`grep` built `dist/` for the service key and NVIDIA key → 0 hits.

ID	Check	Control in this system	Verify
SEC-15	Client-side-only security	UI hides nothing for safety. Server re-checks type, size, ownership, status.	Call the API with `curl`: wrong file type, not-ready doc, other user's id → all refused.
SEC-16	Missing input validation	Pydantic models. UUID-typed ids. Question 1–500 chars. Magic-byte file check. Size and page caps.	Oversize, non-UUID, empty, wrong type → 422 or refused.
SEC-17	SQL injection	No string-built SQL anywhere. Supabase client and parameterised RPC only. User text goes into parameters. Never interpolate user input into `.or()` or `.filter()` strings.	Grep for f-strings near DB calls. Test question `'; drop table claims; --`.
SEC-18	NoSQL injection	No NoSQL store. For JSONB: never pass user-supplied objects as filters or operators. Filters built from fixed columns.	Send `{"$ne": null}` as an id → 422.

Other security items from the earlier screenshots (tenant isolation, prompt injection, rate limits, timeouts, fail-closed, CORS, XSS, redirects, enumeration) are tracked in the Build Guide section 11.

Part C: App / website flow

1. Routes

Route	Screen	Auth	Pri
`/login`, `/signup`	Sign in / sign up	public	P0
`/new`	New claim: upload + question	user	P0
`/claims`	Claim history (own)	user	P0
`/claims/:id`	Result	user (RLS + ownership)	P0

Route	Screen	Auth	Pri
`/privacy`, `/terms`	Legal pages	public	P0
`*`	Custom 404	public	P0
`/review`, `/review/:id`	Reviewer queue and detail	reviewer role	P1

The Build Guide skipped `react-router-dom` for 3 screens. Legal pages, a 404, and shareable result URLs need routes, so add it (pinned) here.

Route guard in the UI is a convenience only (SEC-15). The server decides.

2. Main journey

```
Landing/Login ─► Sign up (email confirm) ─► /new
   /new: pick policy PDF — pick bill — type question — consent notice —
Analyse
      │
      ├─ Uploading ─► Reading documents ─► Searching policy ─► Checking
evidence
      │        (each step shows done / working / failed)
      ▼
   /claims/:id  Result
      ├─ Covered / Partially covered / Not covered  → evidence + amounts
      ├─ Documents missing                          → checklist
      └─ Needs human review                         → explanation + next step
   /claims  history ─► open any past result
```

3. Step-by-step

#	User does	System does	Server checks
1	Signs up	Supabase Auth sends confirmation email	email confirmed before login works
2	Signs in	session + JWT in browser	generic error on failure
3	Chooses files, types question	client checks type, size, length (convenience)	repeated server-side
4	Clicks Analyse	upload to `<uid>/<uuid>.<ext>`; insert `documents` rows; call `/process`	RLS insert policy; ownership; status `uploaded`

#	User does	System does	Server checks
5	Waits	backend parses, OCRs, embeds; browser polls status	magic bytes, page cap
6	(automatic)	when both docs `ready`, browser calls `/claims/analyze`	ownership, kind, status, rate limit
7	Reads result	claim row returned, page renders	RLS on read
8	Opens history	list of own claims from Supabase	RLS

4. State machines

Document status

```
uploaded → processing → ready
                └──→ failed (error = exception type only)
```

Only the backend moves a document past `uploaded`. Users have no UPDATE right.

Claim outcome

```
analyze requested
   ├─ evidence sufficient + rules verified → covered | partially_covered |
not_covered
   ├─ insufficient + documents named       → documents_missing
   └─ insufficient, retries used up        → abstain → human_review_required
= true
(any verdict) + low OCR / total mismatch / no verified rule →
human_review_required = true
```

5. Error and edge flows

Situation	What the user sees	What happens
Wrong file type or too big	inline error at the file field, before upload	server also refuses
File unreadable	"We couldn't read this file. Try a clearer scan or a PDF." + retry	document `failed`

Situation	What the user sees	What happens
Processing takes too long (3 min)	"This is taking longer than usual." + retry / contact	polling stops. Reaper marks stale docs failed (P1).
LLM service down	"Analysis is unavailable right now. Try again in a few minutes."	502. No claim row.
Rate limit hit	"You've reached the hourly limit. Try again later."	429
Session expired	redirect to login, return to the same page after	token refresh failed
Other user's claim URL	same 404 page as a missing claim	RLS returns no row
Bill total mismatch	result shows a limitation: "Printed total differs from the sum of lines."	`flags` + `human_review_required`
Abstain	"We couldn't reach a reliable conclusion. A human should review this." + what was checked	no verdict

6. Reviewer flow (P1)

```
/review (queue: human_review_required = true, oldest first)
   → open claim → see evidence + original quotes → choose outcome + write notes → submit
   → audit event written for every view and decision
```

Access goes through reviewer-only API routes, not a broad RLS policy. Cross-user access stays on the server where it can be logged.

7. Security touchpoints in the flow

Step	Items
Sign-up / sign-in	SEC-4
Upload	SEC-8, SEC-15, SEC-16

Step	Items
Process / analyze	SEC-5, SEC-6, SEC-16, SEC-17
Result / history	SEC-6, SEC-12, SEC-14
Reviewer	SEC-5, SEC-9

Part D: UI/UX design brief

1. Objective

Help a stressed person understand, in under a minute, whether a hospital bill is likely to be covered and why, with proof they can check. Calm, plain, honest about uncertainty.

2. Principles

1. **Evidence first.** Every conclusion sits next to its quote and page.
2. **Say "we don't know" clearly.** Abstain and missing-document states are designed screens, not errors.
3. **Never colour alone.** Icon + text + colour for every status.
4. **Numbers are precise.** Mono font, right-aligned, currency symbol, no rounding surprises.
5. **Always preliminary.** The disclaimer is visible on every result.
6. **Phone first.** Many users will be at the hospital.

3. Screens

3.1 Sign in / sign up

Email, password (show/hide toggle), submit. One generic error for any failure. Link to privacy and terms. After sign-up: "Check your email to confirm."

3.2 New claim (`/new`)

Element	Detail
Policy upload	drop zone + button. PDF only. Shows name, size, remove.

Element	Detail
Bill upload	PDF, PNG, JPEG. Camera capture on phones.
Question	labelled textarea, counter (500), example below it
Consent notice	above the button (copy in section 6)
Analyse button	disabled until both files and a valid question
Progress	4-step list below. `aria-live="polite"`. No model reasoning shown.

States: empty · file chosen · uploading · processing · failed (per file, with retry) · rate-limited.

3.3 Result (`/claims/:id`)

Order, top to bottom:

1. Persistent banner: preliminary assessment.
2. Verdict badge + one-line plain explanation.
3. Amounts block (claimed / estimated eligible / estimated not eligible). Hidden when not calculable, replaced by "Not calculable from the evidence supplied".
4. "Why": the `reason` text.
5. Policy evidence cards: clause ID chip, page chip, quote in mono, "supports: ...".
6. Bill evidence rows: item, qty, amount, page, source text.
7. Missing documents checklist (if any).
8. Limitations (low OCR, total mismatch, no verified rule).
9. Actions: new claim, delete documents.

States: loading skeleton · each of the 5 outcomes · not found (generic 404).

3.4 History (`/claims`)

List newest first: date, question (truncated), verdict badge, flag icon if human review. Empty state: "No claims yet" + button to start one.

3.5 Reviewer (P1)

Two-pane on desktop: queue left, claim right. Notes field, outcome select, submit. Clear "you are viewing another user's document" label. Session timeout shorter than normal.

3.6 Legal and 404

Privacy policy, terms (drafted with counsel, not by the model). 404 has a link home and no details.

4. Verdict presentation

Verdict	Label	Colours (text / bg)	Icon	Plain sentence
`covered`	Covered	`#166534` / `#DCFCE7`	CheckCircle2	"Based on the clauses below, this bill appears payable."
`partially_covered`	Partially covered	`#92400E` / `#FEF3C7`	AlertTriangle	"Part of this bill appears payable. See what reduces it."
`not_covered`	Not covered	`#991B1B` / `#FEE2E2`	XCircle	"A clause below appears to exclude this bill."
`documents_missing`	Documents missing	`#1E40AF` / `#DBEAFE`	FileQuestion	"We need more documents to decide."
abstain / review	Needs human review	`#3F3F46` / `#E4E4E7`	UserCheck	"We couldn't reach a reliable conclusion."

5. Visual system

Tokens, fonts, spacing: see the Build Guide section 5.4 (teal `#0B5563` primary, IBM Plex Sans + Mono, 8 px radius, 4/8/16/24/32 spacing). Flat, clinical, generous whitespace. No gradients. Icons from lucide, no emoji. Dense layout only inside evidence cards.

6. Microcopy

- **Banner:** "Preliminary assessment based on the documents you uploaded. Your insurer makes the final decision."

- **Consent (above Analyse):** "Your documents contain health information. They are stored privately in your account. Text from them is sent to an AI service to analyse them. You can delete them at any time." (Update if you self-host the model.)
- **Login error:** "Email or password is incorrect." (same for unknown email)
- **Unreadable file:** "We couldn't read this file. Try a clearer scan or a PDF."
- **Rate limit:** "You've reached the hourly limit. Please try again later."
- **Generic server error:** "Something went wrong. Reference: <request id>."
- **Empty history:** "No claims yet."

Never show raw error text, file paths, model names, or stack traces (SEC-12).

7. Accessibility (WCAG 2.1 AA target)

- Text contrast at least 4.5:1. Check muted text.
- All controls at least 44 px high. Visible 2 px focus ring.
- Labels on every input. Errors tied to fields with `aria-describedby`.
- Status changes announced with `aria-live`.
- Keyboard: full flow without a mouse. Drop zone also a button.
- Tables have headers. Evidence cards are landmarks or lists with names.
- `prefers-reduced-motion` respected. No motion needed to understand anything.
- Zoom to 200% without loss of content.

8. Responsive

Width	Layout
375–639 px	single column, evidence cards stacked, amounts as stacked rows
640–1023 px	single column, wider cards
1024 px+	main column + sticky side summary (verdict + amounts)

Bill evidence table scrolls inside its own container. The page itself never scrolls sideways.

9. Content safety and privacy in the UI

- Quotes and bill text render as plain text. Never as HTML (SEC XSS item 19 in the screenshots).
- No document content in page titles, URLs, or analytics.
- No analytics in MVP, so no cookie banner needed. Add consent if that changes.
- The Supabase session token sits in browser storage by default. Reduce the risk with short JWT expiry, a strict CSP, and no `dangerouslySetInnerHTML`. Do not store document text in browser storage.
- Result pages are behind login. Add `no-store` cache headers on API responses.

10. Launch must-haves (from the screenshots)

Privacy policy · Terms · HTTPS · favicon + page title · custom 404 · form validation · spam protection (rate limit) · mobile layout · contrast fixes · no frontend secrets. Skip for MVP: sitemap, social preview image, UTM, image compression, dark-mode toggle, site search.

Part E: Backend schema (v2)

Supersedes the SQL in Build Guide 3.2. Changes: least-privilege grants (browser no longer reads chunks or bill items), column-level insert grant, new tables for reviews and audit, claims survive document deletion with evidence scrubbed.

1. Principles

- RLS on every table (SEC-6, SEC-7). Tables with no browser access have RLS on and no policies.
- Browser roles get the smallest grant that works. Everything else goes through FastAPI.
- No `SECURITY DEFINER`. Functions that take a user id are callable only by `service_role`.
- Roles never come from `user_metadata`. Reviewer role = row in `reviewers`, checked server-side.
- No document text in `audit_events`, `trace`, or `error`.

2. Tables

```
create extension if not exists vector with schema extensions;

create table public.documents (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  kind text not null check (kind in ('policy','bill')),
  storage_path text not null unique,
  ext text not null check (ext in ('pdf','png','jpg','jpeg')),
  size_bytes bigint,                -- set by backend from the real blob,
never by the browser
  page_count int,
  status text not null default 'uploaded'
    check (status in ('uploaded','processing','ready','failed')),
  error text,                       -- exception type only
  flags jsonb not null default '[]', -- e.g. total mismatch, low OCR
  created_at timestamptz not null default now(),
  processed_at timestamptz,
  delete_after timestamptz not null default (now() + interval '30 days')  --
placeholder: decide N with counsel
);
create index on public.documents (user_id, created_at desc);
create index on public.documents (delete_after);

create table public.policy_chunks (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  document_id uuid not null references public.documents(id) on delete
cascade,
  page int not null,
  clause_id text,
  content text not null,
  ocr_conf real not null default 1,
  embedding extensions.vector(384),
  fts tsvector generated always as (to_tsvector('english', content)) stored
);
create index on public.policy_chunks using hnsw (embedding
extensions.vector_cosine_ops);
create index on public.policy_chunks using gin (fts);
create index on public.policy_chunks (user_id, document_id);

create table public.bill_items (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  document_id uuid not null references public.documents(id) on delete
```

```
cascade,
  page int not null,
  item text not null,
  quantity numeric not null default 1,
  unit_price numeric,
  amount numeric not null,
  currency text not null default 'INR',
  source_text text not null,
  ocr_conf real not null default 1
);
create index on public.bill_items (user_id, document_id);

create table public.claims (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  policy_id uuid references public.documents(id) on delete set null,
  bill_id uuid references public.documents(id) on delete set null,
  question text not null check (char_length(question) between 1 and 500),
  verdict text check (verdict in
('covered','partially_covered','not_covered','documents_missing')),
  human_review_required boolean not null default false,
  result jsonb,                       -- validated verdict object
  trace jsonb not null default '[]', -- CRAG steps: attempt, query, flags. No
document text.
  model_id text,
  llm_calls int,
  duration_ms int,
  scrubbed_at timestamptz,           -- set when evidence was removed after
document deletion
  created_at timestamptz not null default now()
);
create index on public.claims (user_id, created_at desc);
create index on public.claims (human_review_required, created_at) where
human_review_required;

-- P1 tables. Created now so the schema is stable.
create table public.reviewers (
  user_id uuid primary key references auth.users(id) on delete cascade,
  added_at timestamptz not null default now()
);

create table public.claim_reviews (
  id uuid primary key default gen_random_uuid(),
  claim_id uuid not null references public.claims(id) on delete cascade,
  reviewer_id uuid references auth.users(id) on delete set null,
  outcome text not null check (outcome in
```

```
('agree','disagree','needs_more_info')),
  notes text check (char_length(notes) <= 2000),
  created_at timestamptz not null default now()
);

create table public.audit_events (
  id bigint generated always as identity primary key,
  at timestamptz not null default now(),
  actor uuid,
  action text not null,              -- e.g. document.process, claim.analyze,
review.view
  target_type text,
  target_id uuid,
  meta jsonb not null default '{}'   -- counts, status codes. Never document
text.
);
create index on public.audit_events (actor, at desc);
```

3. Access control

```
-- RLS everywhere
alter table public.documents     enable row level security;
alter table public.policy_chunks enable row level security;
alter table public.bill_items    enable row level security;
alter table public.claims        enable row level security;
alter table public.reviewers     enable row level security;
alter table public.claim_reviews enable row level security;
alter table public.audit_events  enable row level security;

-- Start from nothing for browser roles
revoke all on public.documents, public.policy_chunks, public.bill_items,
public.claims,
              public.reviewers, public.claim_reviews, public.audit_events
  from anon, authenticated;

-- Browser may read own documents and claims, and insert a document row (4
columns only)
grant select on public.documents, public.claims to authenticated;
grant insert (user_id, kind, storage_path, ext) on public.documents to
authenticated;

create policy documents_select on public.documents for select to
authenticated
  using ((select auth.uid()) = user_id);
```

```
create policy documents_insert on public.documents for insert to
authenticated
  with check (
    (select auth.uid()) = user_id
    and status = 'uploaded'
    and storage_path like (select auth.uid())::text || '/%'
  );

create policy claims_select on public.claims for select to authenticated
  using ((select auth.uid()) = user_id);

-- policy_chunks, bill_items, reviewers, claim_reviews, audit_events: RLS on,
no policy, no grant.
-- Only the backend (service_role) touches them.
```

Storage

```
insert into storage.buckets (id, name, public, file_size_limit,
allowed_mime_types)
values ('uploads', 'uploads', false, 10485760,
        array['application/pdf','image/png','image/jpeg'])
on conflict (id) do update set
  public = false, file_size_limit = excluded.file_size_limit,
  allowed_mime_types = excluded.allowed_mime_types;

-- No upsert in the app, so INSERT + SELECT is enough. No UPDATE, no DELETE
for users.
create policy uploads_insert on storage.objects for insert to authenticated
  with check (bucket_id = 'uploads'
    and (storage.foldername(name))[1] = (select auth.uid())::text);
create policy uploads_select on storage.objects for select to authenticated
  using (bucket_id = 'uploads'
    and (storage.foldername(name))[1] = (select auth.uid())::text);
```

The bucket mime limit is a first filter. The backend still checks magic bytes (SEC-16).

4. Functions

```
create or replace function public.hybrid_search(
  q text, q_emb extensions.vector(384), uid uuid, doc uuid, k int default 8
) returns table (id uuid, page int, clause_id text, content text, ocr_conf
real, score double precision)
language sql stable security invoker as $$
  with sem as (
```

```
    select id, row_number() over (order by embedding <=> q_emb) as r
    from public.policy_chunks where user_id = uid and document_id = doc
    order by embedding <=> q_emb limit 20
  ), kw as (
    select id, row_number() over (order by ts_rank_cd(fts,
websearch_to_tsquery('english', q)) desc) as r
    from public.policy_chunks
    where user_id = uid and document_id = doc
      and fts @@ websearch_to_tsquery('english', q)
    order by ts_rank_cd(fts, websearch_to_tsquery('english', q)) desc limit
20
  )
  select c.id, c.page, c.clause_id, c.content, c.ocr_conf,
         coalesce(1.0/(60+sem.r),0) + coalesce(1.0/(60+kw.r),0) as score
  from sem full join kw using (id)
  join public.policy_chunks c using (id)
  order by score desc limit k;
$$;

-- Remove document text from claims before a document is deleted (FR-16, US-
10)
create or replace function public.scrub_claims_for_document(doc uuid)
returns void language sql security invoker as $$
  update public.claims
     set result = coalesce(result, '{}'::jsonb) - 'policy_evidence' -
'bill_evidence',
         scrubbed_at = now()
   where policy_id = doc or bill_id = doc;
$$;

revoke execute on function public.hybrid_search(text, extensions.vector,
uuid, uuid, int)
  from public, anon, authenticated;
revoke execute on function public.scrub_claims_for_document(uuid)
  from public, anon, authenticated;
grant execute on function public.hybrid_search(text, extensions.vector, uuid,
uuid, int) to service_role;
grant execute on function public.scrub_claims_for_document(uuid) to
service_role;
```

The keyword branch expects the backend to OR the terms (`room or rent or cosmetic`). See Build Guide 14.2.

Delete flow (backend): call `scrub_claims_for_document` → delete storage object → delete `documents` row (chunks and items cascade; claims keep verdict and amounts, `policy_id` / `bill_id` become null). Verdict and amounts remain. Evidence quotes and bill

text are removed.

Rate limit (FR-15): count the user's claims in the last hour (`select count` on `claims where user_id = $1 and created_at > now() - interval '1 hour'`). Reject with 429 at 10. No extra dependency.

5. Data dictionary

Table	Holds	Written by	Browser access	Personal data?
`documents`	file metadata, status, flags	browser (4 cols, insert), backend (rest)	select own, insert	indirect (links to user)
`policy_chunks`	clause text, embedding	backend	none	yes (policy text)
`bill_items`	bill lines	backend	none	yes (bill text)
`claims`	question, verdict, result, trace	backend	select own	yes (question, evidence)
`reviewers`	reviewer ids	admin by SQL	none	low
`claim_reviews`	outcome, notes	backend (reviewer route)	none	notes may hold personal data
`audit_events`	who did what, when	backend	none	actor id only

6. Verification

Run after every migration.

```
supabase db advisors            # fix all warnings (CLI v2.81.3+)
```

```
-- What can the browser roles touch? Expect: documents (select, insert),
claims (select). Nothing else.
select grantee, table_name, privilege_type
from information_schema.role_table_grants
where table_schema = 'public' and grantee in ('anon','authenticated')
order by grantee, table_name;

-- Act as user B, try to read user A's rows. Expect 0.
begin;
set local role authenticated;
select set_config('request.jwt.claims', '{"sub":"
<USER_B_UUID>","role":"authenticated"}', true);
select count(*) from public.documents where user_id = '<USER_A_UUID>';
select count(*) from public.claims    where user_id = '<USER_A_UUID>';
rollback;
```

Also run Build Guide 6.3 (storage, RPC, bundle grep) with two real accounts.

Part F: Implementation plan

One builder wearing three hats: **FS** (fullstack), **ML** (ML engineer), **SEC** (security gate). Hours are targets, not promises. Stop rule: any failed security gate blocks sharing the link. Under time pressure cut features, never checks.

1. Day 1 (MVP)

Phase	Hours	Hat	Tasks	Exit criteria	Security gate
0 Prep	0	SEC	Accounts, keys, installs, repo, `.gitignore` + `.dockerignore` first, gitleaks pre-commit, two test users	Build Guide section 10 done	SEC-2, 3, 13
1 Foundation	0–2	FS	Supabase project; apply schema v2; bucket limits; FastAPI skeleton with auth dependency, generic error handler, CORS, docs disabled; Nemotron JSON call	`/health` ok; no token → 401; `db advisors` clean; grants query matches section 6	SEC-1, 5, 7, 8, 10, 12

Phase	Hours	Hat	Tasks	Exit criteria	Security gate
2 Ingestion	2–4	ML + FS	`parse.py` (text, OCR, magic bytes, page cap), clause split (with 14.1 fix), bill structuring with verbatim check and sum check, PII redaction, embeddings	fixtures 01 and 06 parse; chunks have page + clause ID; blurry scan flagged	SEC-15, 16
3 Retrieval	4–5.5	ML	`hybrid_search`, OR-ed keyword terms, Recall@5 script on 3–5 labelled questions	right clause in top 5 for the fixture questions	SEC-17
4 Agent	5.5–7.5	ML + FS	CRAG loop (max 2), rule extraction, citation validation, `rules.calc` (14.3 version), verdict mapping, claim insert, rate limit	all 6 fixtures give the expected result; unit tests green; injection fixture unchanged	SEC-5, 6, 17
5 Frontend	7.5–10	FS	Vite app, router, auth, `/new`, `/claims`, `/claims/:id`, legal pages, 404, verdict badges, evidence cards, banner, consent	full journey on a phone-width screen; keyboard-only works	SEC-4, 14, 15
6 Verify	10–11	SEC	Build Guide 6.3 tests; TRD section 12 row by row; `/ponytail-review` on the repo; fix	every SEC row has a recorded pass	SEC-1 to 18
7 Ship	11–12	FS + SEC	Dockerfile, deploy API, deploy frontend, env secrets, Auth URL config, header scan, `/ponytail-audit`, README	live URL; `/docs` 404; `/.env` 404; bundle grep clean	SEC-2, 10, 11, 14

Cut order if behind (first to go): history page polish → amounts block styling → consent copy polish → recall script → fixture 06. Never cut: RLS, citation validation, `Decimal` math, abstain, rate limit, security gates.

2. Days 2–5 (hardening)

Day	Focus	Deliverables
2	Reviewer + data lifecycle	`/reviews` routes with `reviewers` check and audit events; reviewer MFA; delete-document flow with scrub; retention job; stale-`processing` reaper

Day	Focus	Deliverables
3	Accuracy	table-aware parsing for schedules; fuzzy item matching; more rule types (sub-limits, deductibles); package-bill detection → human review
4	Evaluation	30+ labelled cases (real public policy wordings + synthetic bills + expert labels); metrics script (verdict accuracy, OCR amount match, Recall@K, citation precision, abstention quality); set real targets
5	Assurance	full pass of the 18 checks including a manual attack session; restore test of a Supabase backup; load test at expected concurrency; legal text reviewed by counsel; LLM data-terms decision (keep hosted or self-host)

3. Critical path and dependencies

```
Prep → Schema+Auth → Ingestion → Retrieval → Agent → Frontend → Verify → Ship
                          └─ Nemotron model ID + key needed by hour 1
```

Blocking items: NVIDIA key and exact model ID; Supabase project; Tesseract installed; fixtures generated (Build Guide 17.1).

4. Risks to the schedule

Risk	Likelihood	Response
Nemotron JSON unreliable	medium	tighten prompt, lower `max_tokens`, test 2 model IDs, keep 3 retries
supabase-py call shapes differ from the guide	medium	test each call alone in a scratch script before wiring
OCR quality poor on phone photos	medium	accept: flag low confidence → human review; add preprocessing on day 3
Hosted free tier sleeps or limits memory	medium	check plan; embedding model needs RAM; pick a plan that fits
Frontend takes longer than 2.5 h	high	ship plain layout first, polish after gates

5. Rollback

- Backend: keep previous Docker image; redeploy it.
- Schema: every change as a migration file with a down note. Test restore on a copy first.
- Leaked key: rotate immediately in the provider, update host secret store, redeploy. Deleting a commit is not enough (SEC-13).

6. Definition of done (MVP)

- All P0 requirements in PRD section 6 work end to end.
- 6 fixtures pass. Unit tests green.
- Every SEC-1..18 row has a recorded pass (date, who, evidence).
- Launch must-haves in UX brief section 10 done.
- No real user data until: LLM data-terms read, consent notice live, retention decided, counsel review of privacy and terms.

7. Not verified in these docs

- No code run. Supabase CLI behaviour, `supabase-py` call shapes, and the column-level grant behaviour with the Data API should be tested on a scratch project first.
- Bucket columns (`file_size_limit`, `allowed_mime_types`) and Auth settings (password rules, leaked-password protection) depend on current Supabase versions and plan. Check the dashboard.

- Limits (10 MB, 30 pages, 10 per hour, 30-day retention) are proposals.
- Legal points (DPDP Act, privacy text) need a qualified professional.